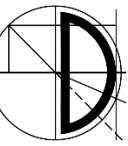


Grundlagen der Programmierung

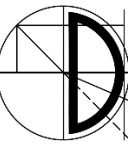
Kapitel 2: Erste Schritte

Prof. Dr. Samuel Kounev, Daniel Grillmeyer M.Sc.

- Alle Materialien (inkl. Vorlesungs- und Übungsfolien, Vorlesungs- und Übungsaufzeichnungen, Übungsblätter, Programmieraufgaben) in diesem Kurs werden nur für Ihre persönliche Nutzung bereitgestellt. Das Teilen oder Weitergeben der Inhalte an Dritte ist generell untersagt!
- Vorlesungsmaterialien von Prof. Dr. Detlef Seese wurden als Basis verwendet
- Unterstützung bei der technischen und inhaltlichen Gestaltung des Vorlesungsmaterials leisteten: Martin Sträßer, Daniel Grillmeyer, Jóakim v. Kistowski, Dietmar Ratz, Joachim Melcher, Roland Küstermann, Jana Weiner, Hagen Buchwald, Matthes Elstermann, Oliver Schöll, Niklas Kühl, Tobias Diederich



- Wie erstellt man einfache Java/C-Programme?
- Was sind Grundbestandteile der Sprache?
 - Compiler, Grammatik, Grundelemente der Sprache

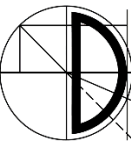


```
public class Kehrwert {  
    public static void main (String[] args) {  
        // Meine Kehrwertberechnung  
        double x, y;  
        x = 5;  
        y = 1 / x;  
        System.out.print("Der Kehrwert von 5 ist ");  
        System.out.println(y);  
        System.out.println("Das war's!");  
    }  
}
```

“Programmkopf”

“Programmrumpf”

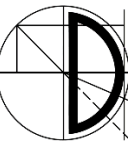
Hier stehen die für
die Funktion des
Programms
relevanten
Anweisungen



```
public class Kehrwert {  
    public static void main (String[] args) {  
        // Meine Kehrwertberechnung  
        double x, y;  
        x = 5;  
        y = 1 / x;  
        System.out.print("Der Kehrwert von 5 ist ");  
        System.out.println(y);  
        System.out.println("Das war's!");  
    }  
}
```

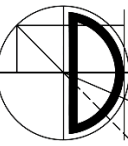
Schlüsselwörter
(**keywords**) sind
von der Sprache
reservierte Wörter

Bezeichner
(**identifiers**) sind
frei wählbare
Namen (u.a. für
Variablen,
Methoden,
Klassen, ...)



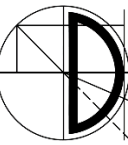
```
public class Kehrwert {  
    public static void main (String[] args) {  
        // Meine Kehrwertberechnung  
        double x, y;  
        x = 5;  
        y = 1 / x;  
        System.out.print("Der Kehrwert von 5 ist ");  
        System.out.println(y);  
        System.out.println("Das war's!");  
    }  
}
```

Manche
Bezeichner
stammen aus
Bibliotheken
und haben
bereits eine
vordefinierte
Bedeutung



```
public class Kehrwert {  
    public static void main (String[] args) {  
        // Meine Kehrwertberechnung  
        double x, y;  
        x = 5;  
        y = 1 / x;  
        System.out.print("Der Kehrwert von 5 ist ");  
        System.out.println(y) ;  
        System.out.println("Das war's!") ;  
    }  
}
```

Kommentare
dienen zur
Dokumentation
des Codes, sie
werden bei der
Übersetzung und
Ausführung des
Programms
ignoriert. In Java
und C/C++
beginnen sie mit
//



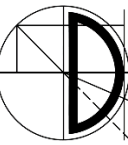
- Seit Java 25: Vereinfachte main-Methode, kompakte Quelldateien und vereinfachtes I/O
- Vereinfachung des Beispiels:

```
public class Kehrwert {  
    public static void main (String[] args) {  
        // Meine Kehrwertberechnung  
        double x, y;  
        x = 5;  
        y = 1 / x;  
        IO.print("Der Kehrwert von 5 ist ");  
        IO.println(y);  
        IO.println("Das war's!");  
    }  
}
```

Kompakte Quelldateien ohne Klassendeklaration geeignet für simple Programme, die nur aus einer Datei bestehen.

Vereinfachte Interaktion mit der Konsole mittels IO-Klasse.

Achtung: PABS läuft aktuell noch auf Java 21.



1. Editieren

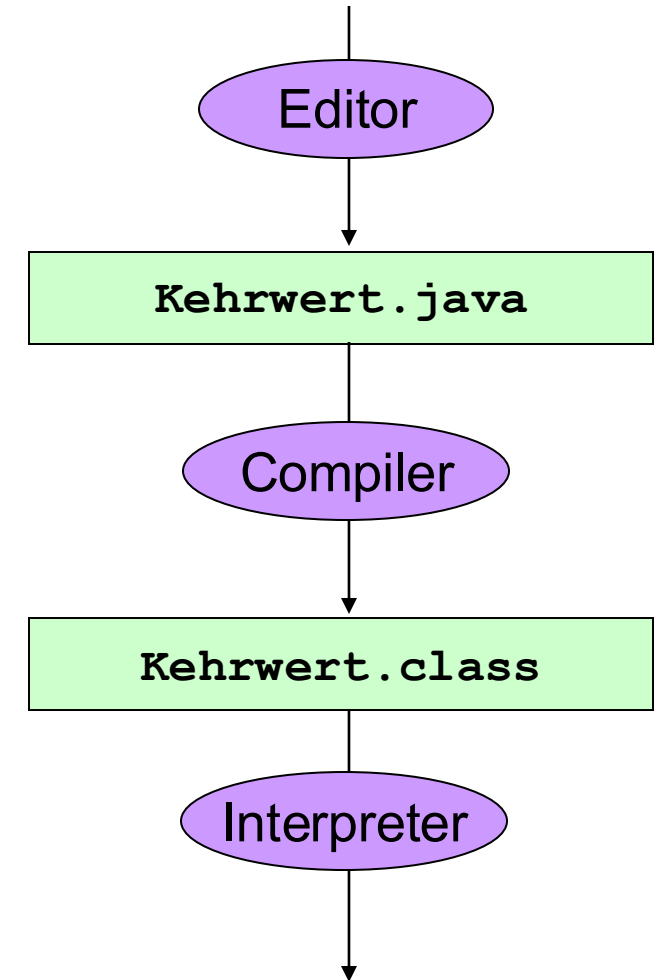
Mit einem Texteditor/Entwicklungsumgebung den Programmtext erstellen und in einer Datei speichern:
z.B. `notepad Kehrwert.java`

2. Übersetzen

Programmtext mit dem Java Compiler in Java-Bytecode übersetzen: `javac Kehrwert.java`
Dabei entsteht die gleichnamige Datei mit Endung `.class`

3. Ausführen

Bytecode mit dem Interpreter ausführen
`java Kehrwert`



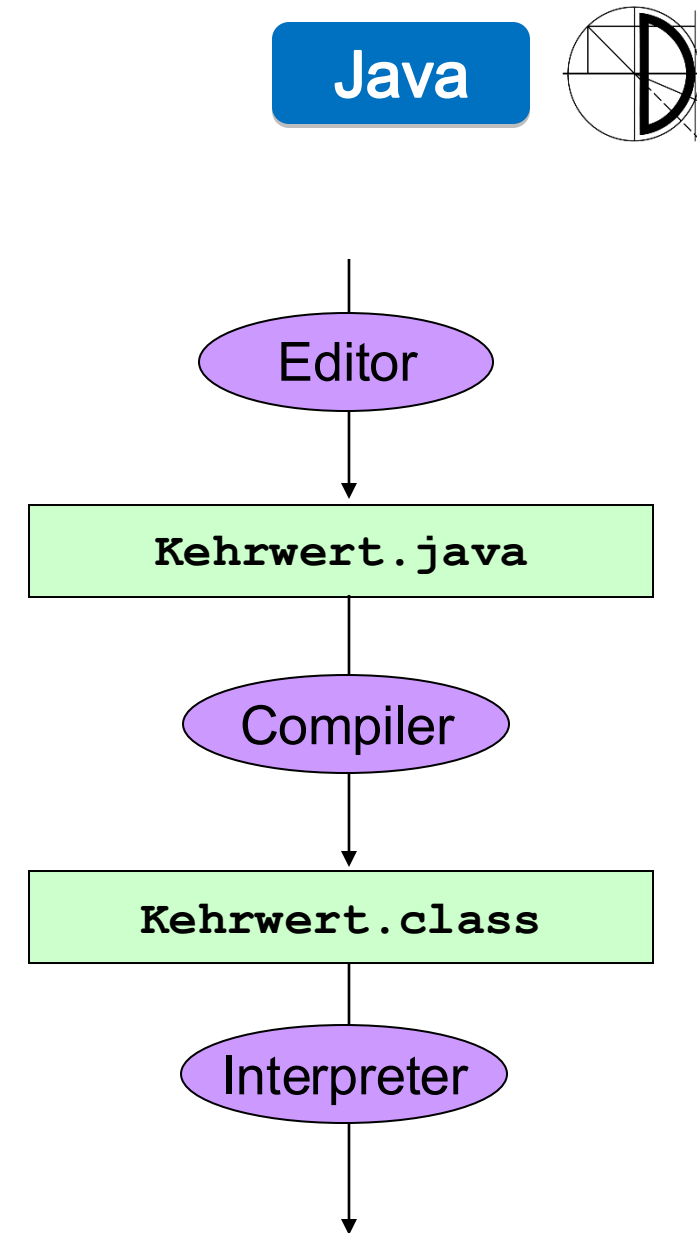
Alternative für kompakte Quelldateien
(seit Java 25):

1. Editieren

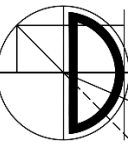
Mit einem Texteditor/Entwicklungsumgebung den
Programmtext erstellen und in einer Datei speichern:
z.B. `notepad Kehrwert.java`

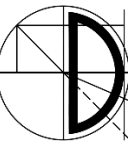
2. Übersetzen und Ausführen in einem Schritt

Implizite Kompilierung zu Java-Bytecode und direkte
Ausführung mit dem Interpreter
`java Kehrwert.java`



Java

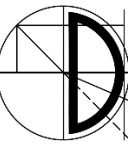




```
#include <stdio.h>
int main()
{
    double x, y;
    x = 5;
    y = 1/x;
    printf("Der Kehrwert von 5 ist %f\n", y);
    printf("Das war's!\n");
    return 0;
}
```

Schlüsselwörter
(keywords)

Bezeichner
(identifiers), zum
Teil in C-
Standardbibliothek
vordefiniert



1. Editieren

Mit einem Texteditor/Entwicklungsumgebung den Programmtext erstellen und in einer Datei speichern:
z.B. `notepad kehrwert.c`

2. Übersetzen

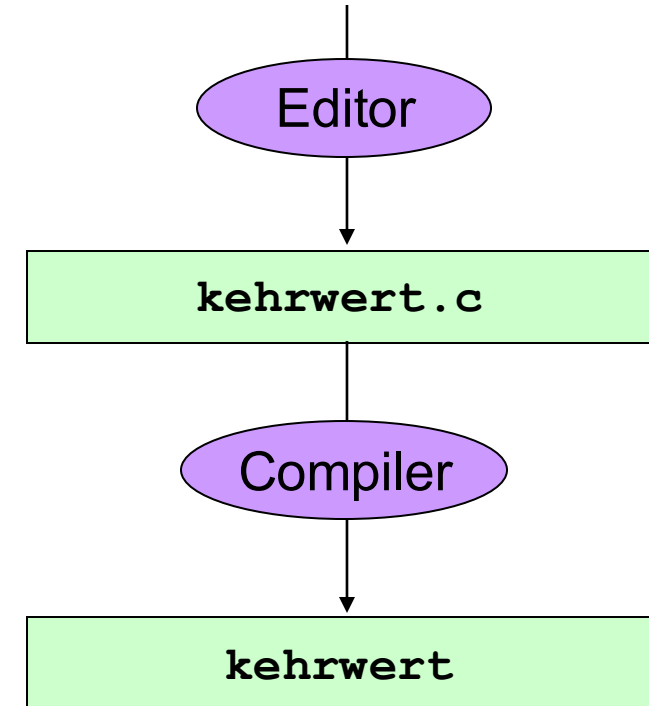
Programmtext mit dem C Compiler in Maschinencode übersetzen:

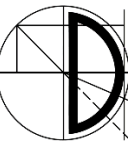
```
gcc -o kehrwert kehrwert.c
```

Dabei entsteht die ausführbare Datei mit Namen
kehrwert

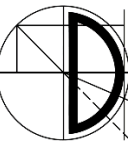
3. Ausführen

Auf Konsole ausführen: `./kehrwert`

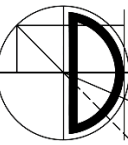




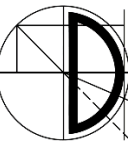
- Der Compiler übersetzt nur "korrekte" Programme
- Die Korrektheit wird anhand der Grammatik überprüft
- **Grammatik**
 - Regeln für die Bildung von "Sätzen" der Sprache, also Programmen
 - Festgelegt durch das Alphabet, die Syntax und die Semantik der Sprache
 - **Alphabet**: definiert den **Zeichenvorrat** der zur Darstellung von Programmen verwendet werden darf
 - **Syntax**: regelt, welche Zeichenfolgen des Alphabets **zulässige Sätze** (Programme) der Sprache bilden.
 - **Semantik**: beschreibt die **Bedeutung** der einzelnen Sprachelemente und die Beziehungen zwischen ihnen, wodurch die Bedeutung des Programms festgelegt wird.



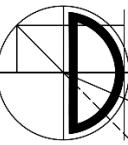
- Jedes Programm ist prinzipiell eine Folge von Zeichen aus dem Zeichenvorrat der jeweiligen Programmiersprache
- Zeichenvorrat von Java, C und C++:
 - **Buchstaben (letters)**
 - Lateinische Groß- und Klein-Buchstaben (wie im ASCII- und ISO-Latin-1-Code)
 - Der Unterstrich (**underscore**): `_`
 - Das Dollarzeichen: `$`
 - Weitere Buchstaben aus dem Unicode (z.B. griechische Buchstaben)
 - **Ziffern (digits)**
 - Lateinische Ziffern 0 bis 9 (wie im ASCII- und ISO-Latin-1-Code)
 - Weitere Ziffern aus dem Unicode (z.B. thailändische Ziffern)



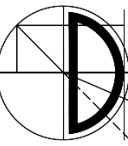
- Whitespace-Zeichen (unsichtbare Zeichen)
 - das Leerzeichen (Blank) (space character)
 - das Zeilenendezeichen (end-of-line character)
 - das Tabulatorzeichen (tab character)
 - das Seitenvorschubzeichen (form feed / page break / new page character)
- Sonderzeichen für Satzzeichen (Interpunktionszeichen)
() { } [] ; , . (punctuation marks)
- Sonderzeichen für Operatoren
= > < ! ~ ? : & | + - * / ^ %
- Sonderzeichen für Ersatzdarstellungen \ (backslash)
- Sonderzeichen für Anführungszeichen
' „ (single resp. double quotes / quotation marks)



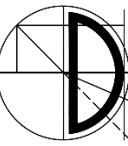
- Der Compiler
 - muss zunächst die Folge von Zeichen scannen
 - sucht darin Gruppen von Zeichen
 - indem nach **Trennern** gesucht wird
 - weitere Zeichen zwischen zwei Trennern bilden ein sogenanntes **lexikalisches Element (Token)**, ein Wort der Sprache
- Mögliche Trenner
 - Whitespace-Zeichen (siehe oben)
 - Kommentare (Zeilenkommentare mit // oder Blockkommentare mit /* oder /**)
 - Satzzeichen (z.B. das Anweisungsendzeichen ;)
 - Operatoren



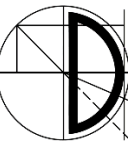
- **Bezeichner** (**identifiers**)
 - beliebig lange Zeichenfolge aus Buchstaben und Ziffern, beginnend mit einem Buchstaben (Unterscheidung Groß- und Kleinschreibung)
- **Schlüsselwörter / Wortsymbole** (**keywords**)
 - reservierte Wörter mit vordefinierter Bedeutung. Können **nicht** als Bezeichner verwendet werden.
- **Literale** (**literals**): repräsentieren konstante Werte von Datentypen
 - Ganzzahl- (**integer**) oder Gleitkomma-Werte (**floating point number**)
 - logische Werte (`true` und `false`)
 - Zeichen-Werte (**char**) (in zwei '-'Zeichen eingeschlossen)
 - Zeichenkettenwerte (**string**) (in zwei '"'Zeichen eingeschlossen)
 - die Nullreferenz (`null`, später in der Vorlesung)



- **Satzzeichen / Interpunktionszeichen** (punctuation marks)
· , ;) (} {] [
- **Operatoren**

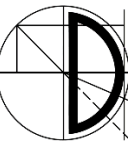


- Werden durch Operatorzeichen dargestellt
- Können aus mehreren Zeichen bestehen, z. B.: `+` `&&` `>>>` `>>>=`
- Operatoren sind sowohl Trenner als auch Tokens
- Die Zeichensequenz `a+b, c-d` besteht für den Compiler aus folgenden lexikalischen Elementen
 - Bezeichner `a`, `b`, `c` und `d`
 - Operatoren `+` und `-`
 - Satzzeichen `,`
- Sinnvoll für Lesbarkeit von Programmen sind zusätzliche Trennzeichen
 - In diesem Beispiel: `a + b, c - d`

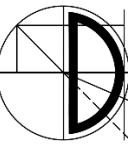


<code>abstract</code>	<code>assert</code>	<code>boolean</code>	<code>break</code>	<code>byte</code>
<code>case</code>	<code>catch</code>	<code>char</code>	<code>class</code>	<code>const</code>
<code>continue</code>	<code>default</code>	<code>do</code>	<code>double</code>	<code>else</code>
<code>enum</code>	<code>extends</code>	<code>final</code>	<code>finally</code>	<code>float</code>
<code>for</code>	<code>goto</code>	<code>if</code>	<code>implements</code>	<code>import</code>
<code>instanceof</code>	<code>int</code>	<code>interface</code>	<code>long</code>	<code>native</code>
<code>new</code>	<code>package</code>	<code>private</code>	<code>protected</code>	<code>public</code>
<code>return</code>	<code>short</code>	<code>static</code>	<code>strictfp</code>	<code>super</code>
<code>switch</code>	<code>synchronized</code>	<code>this</code>	<code>throw</code>	<code>throws</code>
<code>transient</code>	<code>try</code>	<code>var</code>	<code>void</code>	<code>volatile</code>
<code>while</code>				

(Hinweis: `true`, `false` und `null` werden zwar vom Compiler ähnlich wie die Schlüsselwörter behandelt, sind aber technisch gesehen eigentlich Literale)



=	<	>	!	~	?:		
==	!=	<=	>=	&&			
++	--	+	-	*	/	%	
&		^					
<<	>>	>>>	<<=	>>=	>>>=		
+=	-=	*=	/=	&=	=	^=	%=



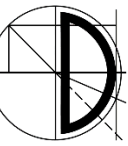
```
public class MitStil {  
    public static void main (String[] args) {  
        int a, b;  
        a = 2;  
        b = 3;  
        a = a * b;  
        b = a % b;  
        System.out.println(a) ;  
    }  
}
```



```
public class ohnestil {public static  
void main (String[ ]args){int A,b;A  
=2;b=3;A=A*b;b=A%b;System.out.println  
(A) ;}}
```



One last thing...



Schon gewusst, dass das Wort `goto` in Java reserviert ist, aber nicht verwendet wird?

Die originale Java Virtual Machine von James Gosling, dem Erfinder der Programmiersprache Java, unterstützte noch das Schlüsselwort, mit dem man zu konkreten Stellen im Code springen konnte. Es wurde allerdings relativ schnell abgelöst, da solche Codesprünge besser mit Methoden (führen wir später ein) realisiert werden können. Das Schlüsselwort `goto` bleibt allerdings nach wie vor reserviert.



The image features a background of a classical building facade with columns and arches, likely a part of the Julius-Maximilians-Universität Würzburg. On the left, there is a blue-tinted area with a white logo consisting of a stylized 'J' and 'M' forming a square. To the right of the logo, the text 'Julius-Maximilians-' is in a smaller font, and 'UNIVERSITÄT WÜRZBURG' is in a larger, bold, white font.

Julius-Maximilians-
**UNIVERSITÄT
WÜRZBURG**

Fragen?

Vielen Dank für Ihre Aufmerksamkeit